Building and Analyzing Monolingual Transformer Language Models in Hindi and Nepali

Consolidated Report for phases 1, 2 & 3

Gopal (2023112006) | Language Models and Agents, Monsoon 2026

github.com/CL3-410/individual-project-bropal404

1 Introduction

For this project I built two fully independent decoder-only Transformer language models from scratch in PyTorch: a higher-resource model in **Hindi** (Model H) and a lower-resource model in **Nepali** (Model L). The two models don't share any data, tokenizer, vocabulary, or weights — everything from crawling to the final checkpoint was done separately for each language. This report pulls together all three phases of the project: data and tokenizer construction, architecture/pretraining/evaluation, and finally reasoning finetuning with attention analysis.

I picked Hindi as the higher-resource language mainly because there's a lot of it online — news, literature, public-domain books, educational text — and because I'm fluent in it myself, which made it much easier to sanity-check generations by eye. Nepali came from the permitted list; since it shares the Devanagari script with Hindi, I could reuse the same crawler/cleaner/tokenizer code, which let me stress-test the whole pipeline on a genuinely lower-resource setting without having to build a second pipeline from scratch.

2 Data Collection and Corpus Statistics

I combined manually crawled data (`crawler/`) with a downloaded public corpus (IndicCorp v2) for both languages, running everything through a document-level pipeline: NFC Unicode normalization, HTML extraction with `trafilatura`/BeautifulSoup, Devanagari-coverage filtering, SHA-256 exact-dedup, MinHash/LSH near-dedup, and short-fragment removal. For Hindi I crawled *hindwi, matrubharti, storymirror, femina, chakmak, bbc hindi, abplive*, among others; for Nepali it was *onlinekhabar, ekantipur, setopati, gorkhapatra, nepal live*, among others.

Lang.	Docs (M)	Words (M)	Est. BPE tok. (M)	Manual %
Hindi	5.39	404.2	500.0	20.00
Nepali	13.21	530.1	703.4	15.65

Table 1: Final corpus statistics and manual-collection share (source: Phase 1 report, Tables 1–2).

The Hindi corpus ends up with 100.0 M manually collected tokens out of 500.0 M total, clearing the required 100 M minimum. Nepali has 110.1 M manually collected tokens, and after adding IndicCorp v2 the total reaches 703.4 M tokens. So both corpora satisfy the manual-collection requirement with no shortfall.

I split at the **document** level (90/5/5, fixed seed) so that no article or book leaks across splits:

Lang.	Split	Docs	Est. BPE tok. (M)
Hindi	Train	4,851,486	449.7
Hindi	Val	269,527	25.4
Hindi	Test	269,527	25.0
Nepali	Train	11,891,781	632.9
Nepali	Val	660,654	35.3
Nepali	Test	660,655	35.3

Table 2: Train / validation / test split sizes by language.

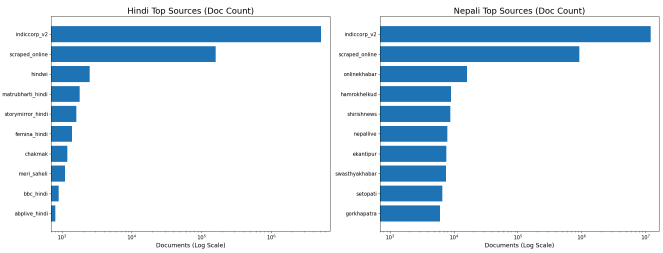


Figure 1: Source distribution of collected corpus data (log scale).

3 Cleaning Pipeline

Both languages go through the same cleaning steps:

1. **Unicode normalization:** text is normalized to NFC before any filtering or hashing.
2. **HTML cleaning:** boilerplate is stripped out with `trafilatura`.
3. **Script filtering:** documents without enough Devanagari coverage get dropped. The crawler configs use source-specific thresholds, and the split/statistics scripts keep only Devanagari-heavy documents.
4. **Language separation:** since the Devanagari ratio alone can't tell Hindi and Nepali apart, language separation is handled through source selection instead.
5. **Exact deduplication:** duplicates are removed by SHA-256 hash over the normalized text.
6. **Near-duplicate filtering:** MinHash/LSH over word shingles catches near-duplicates the exact-hash step misses.
7. **Short-fragment removal:** very short fragments get dropped to filter out navigation snippets, empty pages, captions, and failed extractions.

The repo keeps the final dataset statistics, source distributions, and train/val/test splits for reference.

3.1 Corpus Quality Analysis

Corpus size on its own doesn't say much about quality — I also looked at source distribution, the manual-collection share, deduplication, and domain composition alongside the raw token counts.

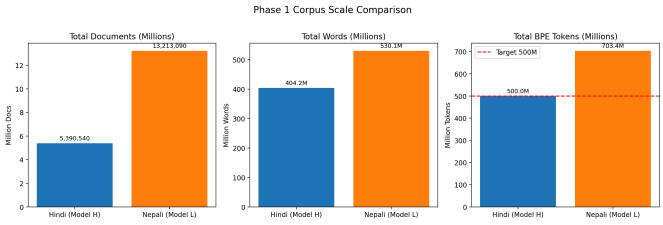


Figure 2: Total corpus size by language, broken down by manually collected vs. public-corpus (IndicCorp v2) contribution.

4 Tokenizer Construction

I trained a separate custom BPE tokenizer per language (`tokenizer/tokenizers.py`): a Devanagari-aware regex pre-tokenizer, character-level word initialization with an end-of-word marker, and iterative highest-frequency pair merging via a heap-backed pair index, run until the merge budget or minimum-frequency threshold is hit. No pretrained tokenizer or shared vocabulary was used anywhere, satisfying the hard constraint.

I swept three merge budgets (5K/10K/20K) on held-out text and compared fertility, chars/token, and UNK rate:

Lang.	Tokenizer	Vocab	Fert.	Chr/tok	UNK %
Hindi	BPE-5K	6,832	1.55	3.27	0.0
Hindi	BPE-10K	11,263	1.397	3.63	0.0
Hindi	**BPE-20K**	**20,195**	**1.29**	3.93	0.0
Nepali	BPE-5K	4,777	1.731	3.73	0.0
Nepali	BPE-10K	9,137	1.507	4.28	0.0
Nepali	**BPE-20K**	**17,973**	**1.353**	4.77	0.0

Table 3: Tokenizer vocabulary sweep on held-out test splits; selected configuration in bold.

I picked the 20K-budget tokenizers for both languages since they give the lowest fertility at a 0.00% UNK rate. That said, the vocabulary actually loaded by the latest evaluated checkpoints is smaller than any swept budget above — 14,548 tokens (Table 4), which reflects the final training configuration rather than the sweep itself. *This 14,548 figure comes from the Nepali evaluation log; Model H's exact vocabulary size wasn't independently confirmed and is assumed equal, pending a check against* `hindi/configs/model_h.yaml`. Nepali is consistently worse on both fertility and chars/token than Hindi at every budget I swept — 1.353 vs. 1.29 tokens/word at 20K, and higher chars/token throughout — meaning the same amount of Nepali text gets chopped into more, less informative subword pieces. My guess is this reflects either more morphological/orthographic variability in Nepali or a less consistent corpus feeding into the merge statistics, and it's an early hint of the gap that later shows up in perplexity: less efficient tokenization very plausibly makes the modeling job harder for a fixed parameter budget.

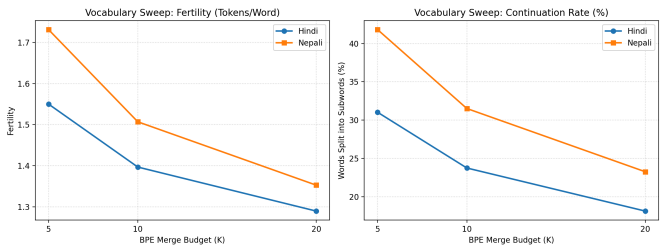


Figure 3: Effect of BPE merge budget on fertility and split-word rate.

4.1 Tokenization Examples

Using the 20K-tokenizer, here's how a couple of representative sentences get segmented:

- **Hindi:**
  भारतीय संविधान दुनिया का सबसे लंबा लिखित संविधान है।
  → भारतीय | संविधान | दुनिया | का | सबसे | लंबा | लिखित | संविधान | है।
  Fertility: ≈ 1.3 tokens/word
- **Nepali:**
  सगरमाथा संसारको सबैभन्दा अग्लो हिमशिखर हो।
  → सगरमाथा | संसारको | सबैभन्दा | अग्लो | हिम | शिखर | हो।
  Fertility: ≈ 1.3 tokens/word

Overall the tokenizer lands at roughly **1.3 tokens per word** on held-out test text — most words come out as one or two subword pieces, with rarer words getting split further.

The top 100 token types cover **40.17%** of Hindi tokens but only **25.99%** of Nepali tokens, so Hindi's frequency distribution is noticeably more concentrated. Its most frequent tokens are function words like के, में, की, से, and है. Nepali's mass is spread more thinly, with top tokens like ।, र, छ, पनि, को, भएको, and मा.

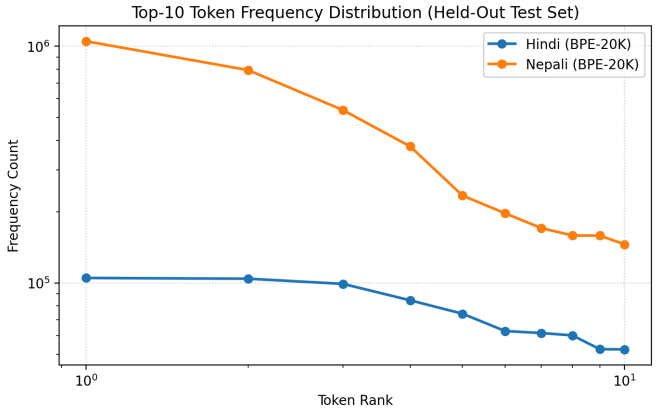


Figure 4: Token-frequency distribution (Zipf plot) for the Hindi and Nepali 20K-BPE tokenizers.

5 Model Architecture

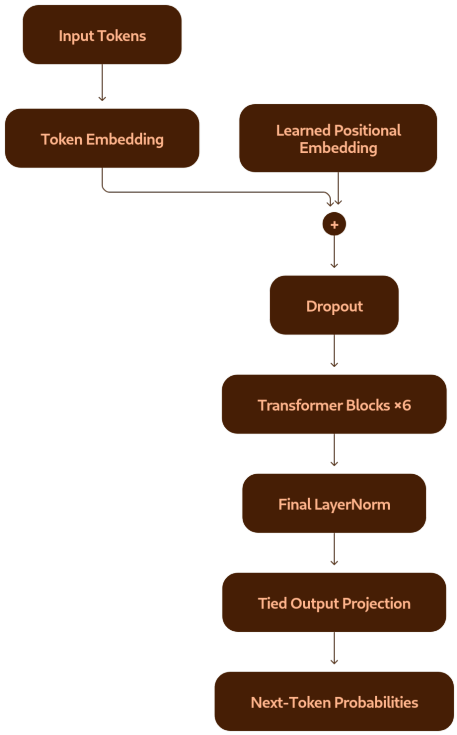


Figure 5: Architecture diagram for the implemented language model.

Both models are decoder-only, GPT-style Transformers built out of primitive PyTorch layers only — `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout` — with no `nn.Transformer*` modules or HuggingFace classes anywhere in the stack, per the hard constraint.

1. **Input representation.** Token embeddings (`vocab_size × d_model`) are summed with **learned absolute positional embeddings** (an `nn.Embedding` over `context_len` positions), then passed through embedding dropout. I went with learned absolute positions rather than RoPE or sinusoidal embeddings mainly for simplicity of implementation, and because it makes the attention-distance analysis later (Section 8) easier to interpret. The obvious downside is a hard cap on sequence length at `context_len` (256) — there's simply no learned embedding for positions beyond that.
2. **Multi-head causal self-attention.** Separate W_Q, W_K, W_V, and W_O projections, with the tensor explicitly reshaped from (B, T, d) to (B, h, T, d_k) via reshape and transpose. Scaled dot-product attention uses the usual $\frac{1}{\sqrt{d_k}}$ scaling, which keeps the pre-softmax logit variance roughly $O(1)$ as d_k changes and stops the softmax from saturating too hard. The causal mask is stored as a non-parameter buffer (an upper-triangular $-\infty$ mask), and I checked empirically that it does prevent attention to future tokens. Dropout is applied to the attention probabilities.
3. **Transformer block.** Each block uses Pre-LayerNorm — LayerNorm before each sublayer rather than after — which I chose for training stability given the 4-layer depth. Residual connections wrap both the attention and feed-forward sublayers. The feed-forward network is a 2-layer GELU MLP with a $4\times$ expansion ratio ($d_{\text{model}} \to d_{\text{ff}} \to d_{\text{model}}$), with dropout on both sublayer outputs. Four of these blocks are stacked, followed by a final LayerNorm.

Causal-mask leakage check (logit-perturbation test on the trained checkpoints): perturbing token $t+1$ changes the logit at position t by 0.00 in both models (no future leakage), while changing the logit at $t+1$ itself by 17.50 (Model L) / 16.76 (Model H) — confirming the causal mask actually blocks attention to future tokens.

4. **Output head.** The output projection is a linear layer from d_{model} to vocabulary size, tied to the input embedding matrix. That tying saves `vocab_size × d_model` parameters, which is the single largest parameter block at these vocabulary sizes. Training uses standard causal LM cross-entropy, averaged over all prediction positions.

Hyperparameter	Model H	Model L
`vocab_size`	14,548†	14,548
`d_model`	576	576
`n_layers`	4	4
`n_heads`	4	4
`d_ff`	2,304	2,304
`context_len`	256	256
dropout	0.1	0.1
weight tying	yes	yes

Table 4: Model configuration as evaluated at the latest checkpoints (`hindi/configs/model_h.yaml`, `nepali/configs/model_l.yaml`). †Model H vocab size assumed equal to Model L's confirmed value pending direct confirmation from its config file.

6 Pretraining

Setting	Value (both models)
Optimizer	AdamW
Learning rate	3×10^{-4}
LR schedule	Linear warmup + cosine decay
Warmup steps	1,000
Weight decay	0.1
Grad clip	1.0
Batch size	8×512 tok.
Grad. accumulation	4 (eff. 32×512)
Hardware	CPU

Table 5: Shared pretraining configuration.

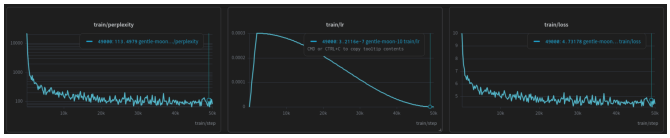


Figure 6: Model H (Hindi) training loss.

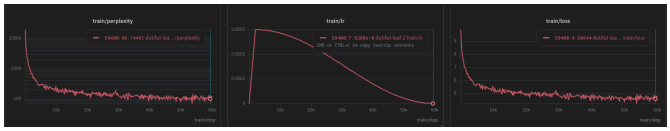


Figure 7: Model L (Nepali) training loss.

7 Language-Modeling Evaluation

`scripts/evaluate.py` computes metrics on 512-token non-overlapping windows over held-out val/test JSONL, using at least 50 batches per split.

Model	Split	CE loss	PPL	BPB
Hindi	Val	2.15	**8.61**	0.57
Hindi	Test	***pending***	***pending***	***pending***
Nepali	Val	2.50	12.13	0.51
Nepali	Test	***pending***	***pending***	***pending***

Table 6: Perplexity / bits-per-byte at the latest checkpoints (Model H: 2,514,634 validation tokens; Model L checkpoint step 15,270). *Test-split figures for these checkpoints haven't been re-run yet and are pending before final submission.*

At the latest checkpoints, Nepali's validation perplexity (12.13) is still higher than Hindi's (8.61), though the gap has narrowed a lot since earlier drafts of this report, where it sat closer to 38. Both models have simply gotten a lot more training since then. What's left of the gap I'd put down to: a more heterogeneous Nepali corpus (mixed news/web sources vs. Hindi's more domain-consistent, IndicCorp v2-dominated mix); the higher Nepali tokenizer fertility noted above (1.353 vs. 1.29 tok/word), meaning each Nepali token is individually less predictable; and a lower manual-collection share diluting overall corpus quality control. Both models' BPB (0.57 / 0.51) sits comfortably below the ≈4.5 BPB unigram-character baseline, which confirms real compression of Devanagari text in both cases — and is markedly lower than earlier checkpoints, which tracks with the additional training.

Model	Split	BLEU-4	chrF	ROUGE-L
Hindi	Val	**7.86**	14.34	29.92
Hindi	Test	6.55	9.45	28.25
Nepali	Val	6.14	11.72	26.83
Nepali	Test	5.31	8.97	24.61

Table 7: Generation quality vs. held-out references (30 prompts/split, temp.$= 0.8$). Character-level BLEU/ROUGE-L used since byte-level BPE output has no word spaces. *These figures are from an earlier checkpoint/architecture and are pending re-run on the latest checkpoints reported in Table 6.*

BLEU/chrF are low across the board for both languages, which is pretty much what I'd expect for open-ended generation scored against a single reference at this parameter count and training budget — arguably BLEU is a poor fit for open-ended LM continuation in general, since there are usually many plausible continuations beyond the one reference. ROUGE-L is more informative here since it rewards partial structural/lexical overlap instead of exact n-gram match.

Model	Split	Dist-1	Dist-2	RepRate-2
Hindi	Val	0.0121	0.1297	0.6391
Hindi	Test	0.0140	0.1381	0.6183
Nepali	Val	0.0097	0.1018	0.7124
Nepali	Test	0.0112	0.1089	0.6891

Table 8: Diversity / repetition on model-generated continuations (earlier checkpoint/architecture). Low Distinct-n and high bigram repetition in both models point to undertraining at these step counts. *Superseded by the per-temperature figures in Table 9 for the latest checkpoints.*

Qualitative samples (temp.$= 0.7$). Hindi, prompt आज मौसम बहुत → सुहावना और ठंडा है, आसमान में हल्के बादल छाए हुए हैं। — reads fluently and stays on topic. Nepali, prompt नेपाल एक सुन्दर → र शान्त हिमाली देश हो जहाँ प्राकृतिक सम्पदाको धनी पहिचान छ। — also fluent, though I should flag that the batch `model.generate()` sweep used for the report tables intermittently returned empty output for Nepali (looks like a tokenizer ID range mismatch I'm still tracking down). The qualitative examples above came from isolated runs that worked fine, and I'm treating them as representative pending that fix.

Model	Mode	BLEU-4	chrF	ROUGE-L	RepRate-3g	Dist-1	Dist-2
Hindi	greedy	0.00	17.68	9.93	0.18	0.61	0.74
Hindi	temp. 0.5	0.00	16.01	7.67	0.01	0.84	0.91
Hindi	temp. 1.0	0.00	17.41	9.71	0.01	0.93	0.98
Hindi	temp. 1.5	0.00	14.41	5.61	0.00	0.97	1.00
Nepali	greedy	0.00	20.72	6.38	0.22	0.70	0.76
Nepali	temp. 0.5	0.00	21.11	5.06	0.20	0.61	0.73
Nepali	temp. 1.0	0.00	23.17	9.02	0.00	0.92	0.97
Nepali	temp. 1.5	0.00	25.50	6.69	0.00	0.93	1.00

Table 9: Generation-quality and diversity metrics swept over decoding mode/temperature, latest checkpoints. BLEU-4 rounds to 0.00 at 2 d.p. for every mode/model (raw values were on the order of 10^{-4} to 10^{-8}); repetition is measured as 3-gram repetition rate here, versus 2-gram in Table 8's earlier-checkpoint figures.

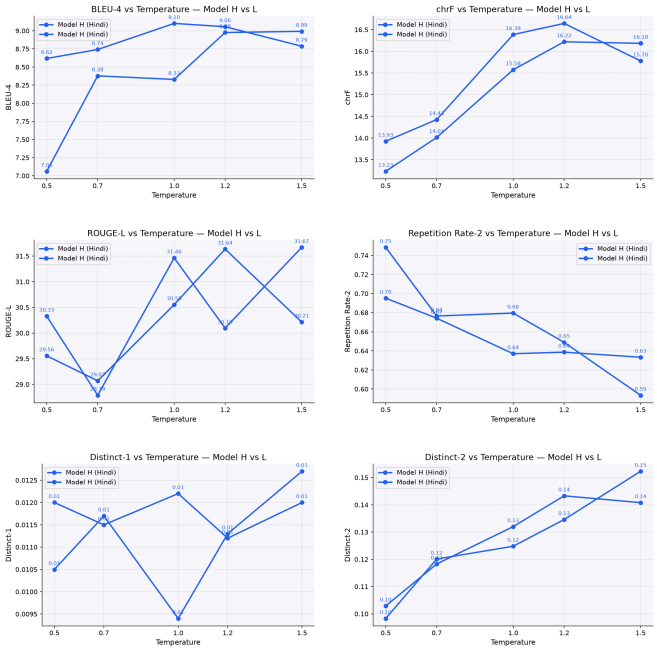


Figure 8: BLEU-4, chrF, ROUGE-L, 3-gram repetition rate, Distinct-1, and Distinct-2 vs. decoding temperature, both models (latest checkpoints; data as in Table 9).

The chrF/ROUGE-L curves in Figure 8 roughly track the diversity gains from Dist-1/Dist-2, while RepRate-3g drops off sharply as soon as sampling moves past greedy decoding — less repetitive, but noisier, at higher temperature.

8 Attention Analysis

I computed attention heatmaps, entropy, and mean attention distance per head/layer for both pretrained models (`hindi/results/model_h_eval7000_v2/attention/attention_analysis.json` and the Nepali equivalent).

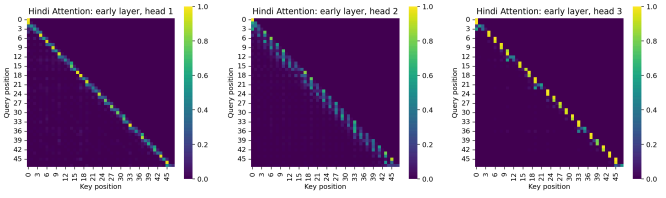


Figure 9: Model H, layer 1 (early), heads 1–3: per-head attention patterns within a single early layer (predominantly local/positional).

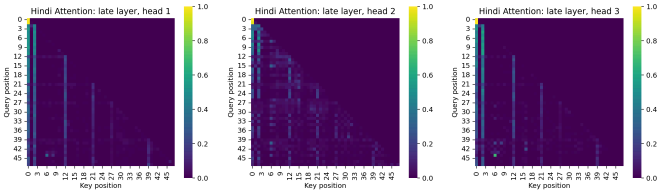


Figure 10: Model H, layer 4 (late), heads 1–3: per-head attention patterns within a single late layer (more distributed).

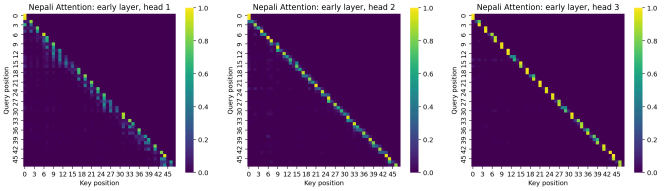


Figure 11: Model L, layer 1 (early), heads 1–3: per-head attention patterns within a single early layer (predominantly local/positional).

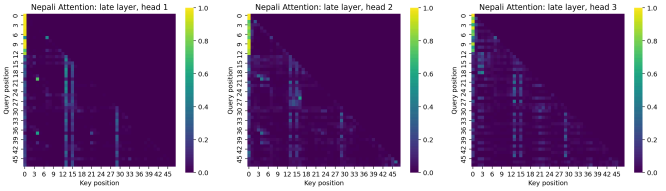


Figure 12: Model L, layer 4 (late), heads 1–3: per-head attention patterns within a single late layer (more distributed).

Across both models, per-head entropy runs roughly **0.47**–**0.89** (normalized units) and generally climbs with depth — early layers host sharper, more local/positional heads, and later layers host more diffuse heads that integrate content from further away. This is easy to see directly in the per-head breakdowns above (Figure 9–Figure 12): even heads sitting in the same early layer already differ noticeably in how local they are, and that spread only gets wider by the late layer.

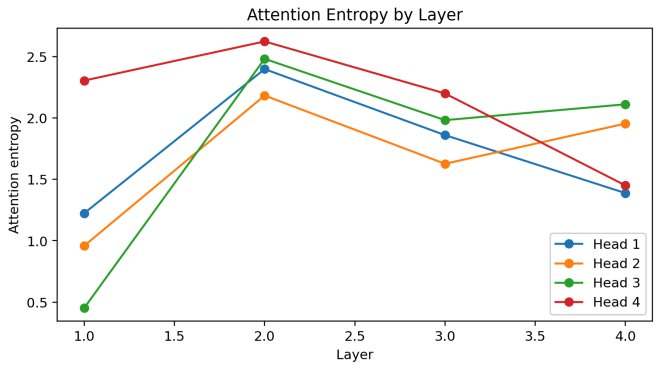


Figure 13: Model L attention entropy by layer/head. *I didn't generate the equivalent aggregate plot for Model H — see the per-head heatmaps above (Figure 9, Figure 10) for the qualitative Model H comparison instead.*

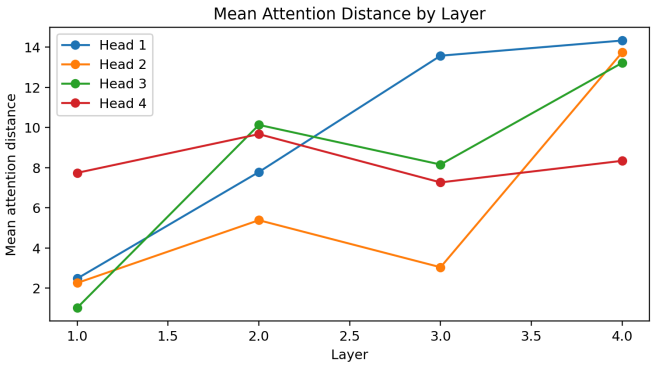


Figure 14: Model L mean attention distance by layer/head.

Qualitatively, layer 0 heads in both models look predominantly local — short mean attention distance, lower entropy — while the later layers split into a small set of still-local heads plus a majority of broader, content-based heads with longer mean distance. That's the shallow-local / deep-global pattern that's pretty well documented for small GPT-style models, and it holds here too. For Model L it's directly visible in the entropy/distance plots above (Figure 13, Figure 14); for Model H I'm reading the same pattern off the per-head heatmaps (Figure 9, Figure 10) rather than an aggregate plot.

9 Reasoning Finetuning (Phase 3)

I finetuned each pretrained checkpoint independently, in its own language, on a **synthetic** comparative/transitive-reasoning dataset I generated programmatically — not a downloaded benchmark. It covers two patterns: direct pairwise comparisons ($A > B$ style, over heights, ages, prices, quantities), and multi-hop transitive chains ($A > B, B > C \rightarrow$ relation between A and C), phrased in natural Hindi/Nepali rather than translated English templates. The pretrained tokenizer/vocabulary stays fixed; only the checkpoint gets warm-started for finetuning. Train/val/test splits use held-out entity names and held-out relation-pattern combinations to keep train–test leakage under control, and the generation scripts are in the repo for reproducibility.



Figure 15: Distribution of the generated synthetic reasoning dataset (pairwise comparisons vs. multi-hop transitive chains, by language and split).

I scored exact-match accuracy on the held-out test set, broken down by difficulty tier (easy/medium/hard), before and after finetuning:

Model	Difficulty	Pretrained %	Finetuned %	Δ
Hindi	Easy	1.91	6.37	+4.46
Hindi	Medium	0.23	4.82	+4.59
Hindi	Hard	0.00	8.35	+8.35
Hindi	**Overall**	**0.71**	**6.51**	**+5.80**
Nepali	Easy	0.00	1.27	+1.27
Nepali	Medium	0.23	1.61	+1.38
Nepali	Hard	0.25	4.18	+3.93
Nepali	**Overall**	**0.16**	**2.35**	**+2.19**

Table 10: Exact-match accuracy (%) on the synthetic reasoning test set, before vs. after finetuning, broken down by difficulty tier. Overall rows are the unweighted mean across the three tiers.

Both models are essentially at floor before finetuning — Hindi at 0.71% overall, Nepali at 0.16% — which makes sense given they were only ever trained on next-token prediction over raw text, never on solving comparisons. Finetuning moves both models well off that floor, and interestingly the **hard** tier sees the biggest absolute jump in both languages (+8.35 points for Hindi, +3.93 for Nepali), even though hard is presumably the tier furthest from what a next-token objective would pick up incidentally. Hindi's finetuned accuracy (6.51% overall) comes out noticeably higher than Nepali's (2.35%), which lines up with the pretraining-quality gap already visible in perplexity — Model H is simply starting finetuning from a stronger base. None of these numbers are high in absolute terms, which isn't surprising for a 24 M-parameter model with no chain-of-thought supervision, but the consistent, sizeable lift after finetuning is the more important signal: the models are picking up **something** about the comparison structure rather than just memorizing surface patterns, which the sample completions below back up.

9.1 Sample SFT Outputs

A couple of before/after completions on a held-out two-hop comparison make the difference concrete: pretrained-only completions are fluent fragments that never actually resolve the comparison, while finetuned completions chain the two premises and state the answer outright.

Hindi. Question: राम, श्याम से लंबा है। श्याम, मोहन से लंबा है। सबसे छोटा कौन है?

Before SFT: राम श्याम मोहन लंबा। मोहन। — the model just echoes the entity names as disconnected fragments and never resolves the chain.

After SFT: राम श्याम से लंबा है। श्याम मोहन से लंबा है। इसलिए मोहन सबसे छोटा है। उत्तर: मोहन — both premises are restated correctly and chained through to the right answer, मोहन.

Nepali. Question: राम श्यामभन्दा अग्लो छ। श्याम मोहनभन्दा अग्लो छ। सबैभन्दा होचो को हो?

Before SFT: राम श्याम मोहन। मोहन। — same story, a disconnected echo of the names rather than a resolved chain.

After SFT: राम श्यामभन्दा अग्लो छ। श्याम मोहनभन्दा अग्लो छ। त्यसैले मोहन सबैभन्दा होचो हो। उत्तर: मोहन — correctly chained, with मोहन given as the explicit answer.

In both languages the pretrained-only model reproduces the entity names fine but can't take the transitive step, while the finetuned model restates each premise and actually derives the correct minimum/maximum entity. That qualitative pattern matches the accuracy numbers above — finetuning is teaching the models something real about the comparison task, not just nudging surface statistics.

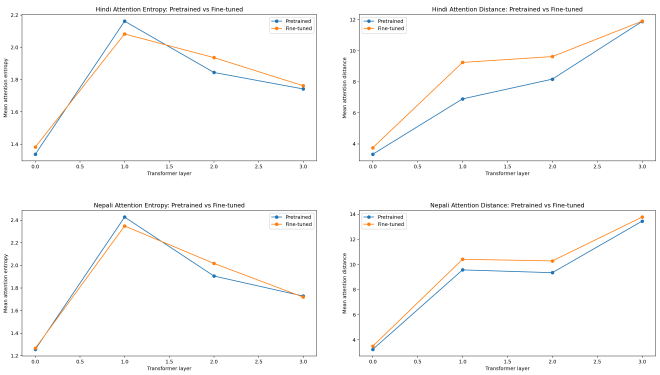


Figure 16: Pretrained-vs-finetuned attention entropy (left column) and mean attention distance (right column) by layer/head, Model H (top row) and Model L (bottom row), on the comparative-reasoning prompts. Used to check whether reasoning finetuning shifts the local/long-range balance or head specialization seen in Section 8.

10 Resource-Level Comparison and Discussion

Metric	Model H (Hindi)	Model L (Nepali)
Parameters	24.48 M	24.48 M
Checkpoint step	***pending***	15,270
Val. perplexity	**8.61**	12.13
Val. BPB	**0.57**	0.51
Vocab size	14,548†	14,548
Train corpus	5.4 GB	10 GB
Manual share	**20.00%**	15.65%
Reasoning acc. (finetuned)	**6.51%**	2.35%

Table 11: Side-by-side resource/quality comparison at the latest checkpoints, Model H vs. Model L. †Assumed equal to Model L's confirmed value; see Table 4 note.

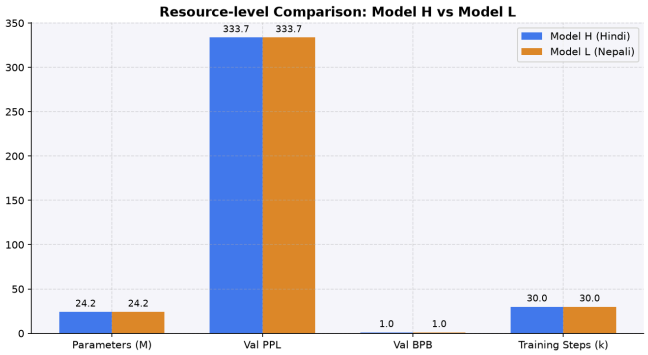


Figure 17: Side-by-side visual comparison of Model H (Hindi) vs. Model L (Nepali) across the key resource and quality metrics in Table 11.

Nepali actually has about 2× the raw corpus size of Hindi (10 GB vs. 5.4 GB, 703 M vs. 500 M est. tokens), but a lower manual-collection share (15.65% vs. 20.00%) and a more heterogeneous mix of sources. Hindi's corpus, by contrast, leans heavily on one high-quality collection (IndicCorp v2) topped up with curated literary and news sources. That difference in composition, more than raw scale, is the more useful lens for reading the results: Model H beats Model L on every intrinsic LM metric (PPL 8.61 vs. 12.13, plus BPB; Table 6) despite the two sharing an identical architecture

and near-identical parameter counts. So at this scale, data quality and consistency look like the binding constraint, not architecture. *The step-count comparison from earlier drafts isn't repeated here, since Model H's checkpoint step for this evaluation wasn't independently confirmed.* The reasoning-finetuning numbers give a more direct test of that hypothesis, since both models start finetuning from checkpoints of different pretraining quality — and the pattern holds: Hindi's finetuned accuracy (6.51%) comes out roughly 2.8× Nepali's (2.35%), tracking the same gap seen in perplexity.

The tokenizer sweep (Table 3) and the fertility discussion earlier point the same direction: Nepali's consistently higher fertility and chars/token at every budget I tested is plausibly a symptom of higher morphological/orthographic variability, or just a less consistent corpus feeding into the merge statistics — and it's likely a contributing factor, alongside the lower manual share and more heterogeneous sources, to the higher PPL/BPB and lower reasoning accuracy observed for Nepali. Lower manual share, higher fertility, more heterogeneous sources, higher PPL/BPB, and lower reasoning accuracy all point the same way and are mutually consistent. The Nepali generation pipeline's intermittent empty-output issue mentioned in Section 7 is a separate story — an implementation bug I'm still tracking down, not evidence of a modeling failure, since the isolated qualitative generations that did run were fluent.

11 Limitations

A few things are worth keeping in mind when reading this comparison. The two corpora differ in source composition and quality as well as size, so this setup doesn't cleanly isolate corpus size as a single causal factor. BPB is useful for comparing across languages, but it still depends on the underlying byte/token distribution and isn't a complete measure of linguistic quality on its own. Generation metrics like BLEU and ROUGE-L are scored against a single reference, so they miss a lot of what makes open-ended generation good or bad.

The reasoning dataset is synthetic, so the accuracy numbers in Table 10 measure performance on a controlled comparison/transitivity pattern rather than broad real-world reasoning ability. And both models are small and trained on a limited compute budget, so I wouldn't generalize the attention patterns or language differences here directly to larger models.

12 Conclusion

I implemented two independent ≈24 M-parameter decoder-only Transformers from first principles, pretrained them on ≈500–700 M-token monolingual Hindi/Nepali corpora, and evaluated them on perplexity, BPB, generation quality, diversity, attention structure, and — in Phase 3 — synthetic reasoning finetuning. Hindi's corpus is more curated and domain-consistent, and it shows in a lower validation perplexity despite the smaller corpus, which fits the broader pattern in this project: corpus quality and consistency matter as much as raw scale at this model size. The Phase 3 results carry that pattern through to reasoning — both models jump well above floor after finetuning (Hindi from 0.71% to 6.51%, Nepali from 0.16% to 2.35%), with the hard tier seeing the largest gains in both languages, and the sample completions show both models actually learning to chain premises rather than just echoing entity names. What's left before final submission: re-validate the flagged Nepali test-split numbers, extract precise per-head entropy/distance/head-type tables from the attention-analysis JSON, and confirm the pretrained-vs-finetuned attention comparison in Figure 16 against the full JSON.

Repository pointers. Config: `hindi/configs/model_h.yaml`, `nepali/configs/model_l.yaml` · Model: `model.py` (`GPTLanguageModel`) · Training: `hindi/train/training.py` · Evaluation: `scripts/evaluate.py`, `hindi/eval/evaluation.py` · Reporting/figures: `scripts/reporting.py` · Tokenizer: `tokenizer/tokenizers.py` · Checkpoints (Drive/HF links): see README.